

# Operating Systems

No.1: Overview of Operating Systems

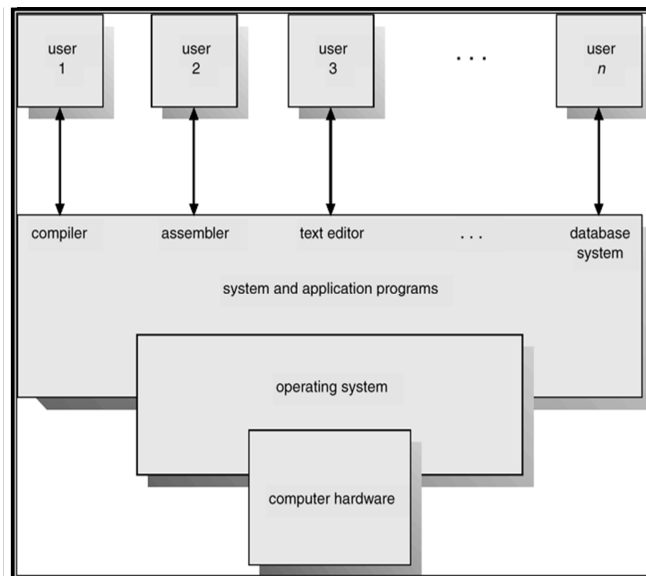
Sarun Intakosum



## Introduction to OS

### What is Operating Systems?

- The software that manages a computer's resources.
- Control who can use the system and how



## Examples of *Mobile OSes*

# Examples of Desktop (Laptop) OSes

# Examples of Unix-Like OSes

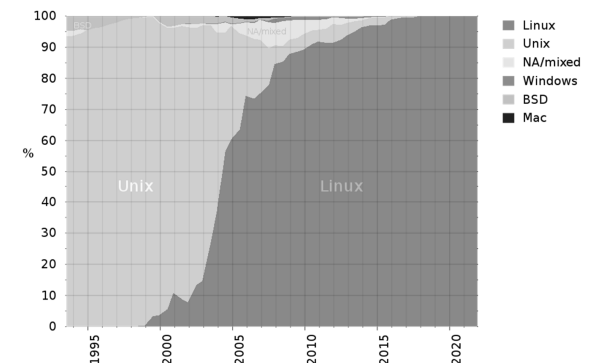
## Examples of Mainframe Operating Systems

- z/OS
- z/Vm
- Linux for System Z



## Examples of Super Computer OSes

- Linux
  - specific changes to the Linux-derivative
  - [http://commons.wikimedia.org/wiki/File:Operating\\_systems\\_used\\_on\\_top\\_500\\_supercomputers.svg](http://commons.wikimedia.org/wiki/File:Operating_systems_used_on_top_500_supercomputers.svg)



## What will we study?

- There are various OSes; what will we study when we say we study OS?
- When we talk about operating systems, we basically mean that an operating system is just its kernel.
- The kernel is responsible for executing the fundamental tasks of an operating system.
- Everything else falls into two categories: system programs and application programs.

## Operating System Goals

- Run user programs and make it easier for users to solve problems.
- Make sure that the computer system is user-friendly.
- Optimize the utilization of computer hardware.

## OS Components

- Memory Manager
- Processor Manager
- Device Manager
- File Manager
- User Command Interface
- Network Manager
- Security Manager
- Each component must perform the following tasks:
  - Monitor its resources
  - Follow the rules for who gets what, when, and how much.
  - Ensure that the resource is allocated at the appropriate time
  - Make sure to deallocate the resource when it is necessary

## How an OS works?

---

# THE BOOT PROCESS

---

Booting – starting a computer by loading the kernel

Power On: When you press the power button, electricity flows through the computer's components, and the Central Processing Unit (CPU) starts executing instructions from the Basic Input/Output System (BIOS) or Unified Extensible Firmware Interface (UEFI) firmware.



## POST (Power-On Self-Test):

- The BIOS/UEFI performs a series of checks on the hardware components to ensure they are functioning correctly.
- This includes testing the memory, CPU, storage drives, and other peripherals.

## Boot loader Selection :

- The BIOS locates the MBR on the boot drive, which contains information about the partitions and the bootloader.
- UEFI locates the ESP, a dedicated partition that contains bootloaders and device drivers.
- UEFI reads the Boot Configuration Data (BCD) store to determine the default bootloader.



# Bootloader Execution:

The BIOS loads and executes the  
bootloader from the MBR.  
UEFI loads and executes the  
selected bootloader from the ESP.

BOOT PROCESS  
4

BOOT PROCESS  
5

# Operating System Load:

The boot loader locates and loads the operating system kernel.

# OS Initialization:

The operating system takes over and  
completes the boot process.

Boot process 6



# Key Differences between BIOS and UEFI

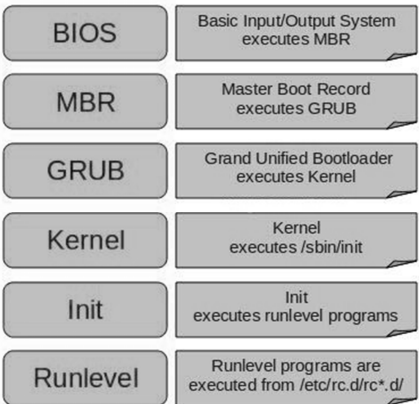
- Boot Mode: BIOS uses 16-bit real mode, while UEFI uses 32-bit or 64-bit mode, allowing for faster operations and larger addressable memory.
- Partition Table: BIOS relies on the Master Boot Record (MBR), which has limitations in partition size and number, while UEFI uses the GUID Partition Table (GPT), offering more flexibility and support for larger drives.
- Bootloader Location: BIOS stores the bootloader in the MBR, while UEFI stores it in the ESP.

# Key Differences between BIOS and UEFI (Cont.)

- Secure Boot: UEFI has the capability to enable Secure Boot, a security feature that verifies the digital signature of the bootloader to prevent unauthorized software from running.
- Boot Speed: Generally, UEFI boots faster than BIOS due to its optimized initialization process and direct access to the bootloader in the ESP.



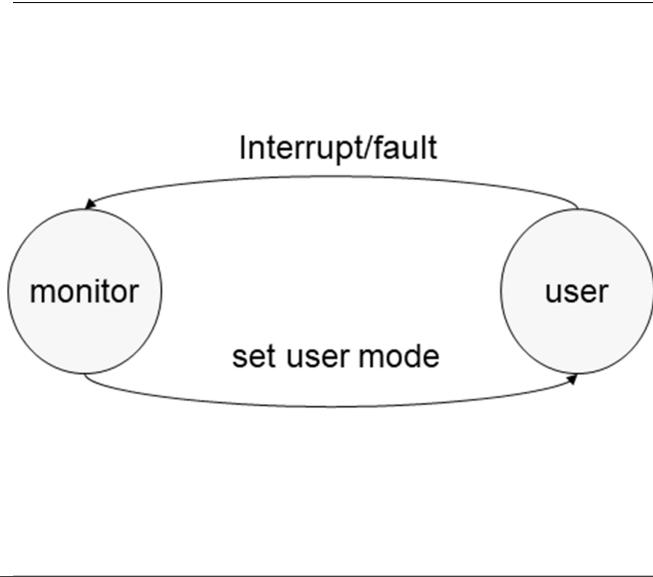
## Linux Boot Process (Legacy BIOS)



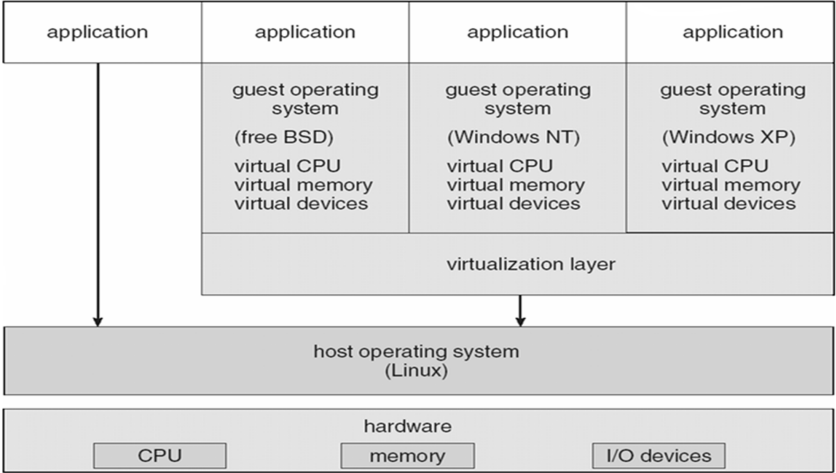
<https://www.thegeekstuff.com/2011/02/linux-boot-process/>

## Dual Mode Operation

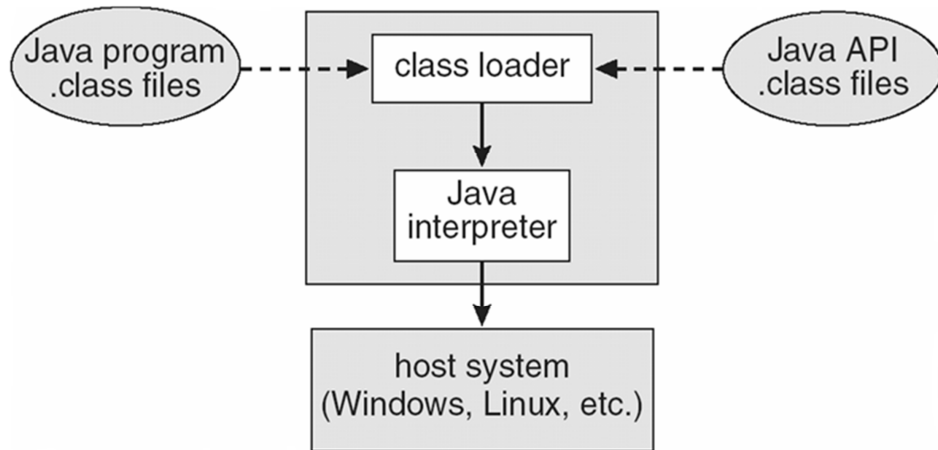
- Mode Bit: A hardware-supported flag indicates the current execution mode (kernel or user).
- The mode bit is set to 0 for kernel mode and 1 for user mode.
- When an interrupt or fault occurs hardware switches to monitor mode.
- Privileged instructions can be used only in monitor mode.



## Virtual Machine



## Java Virtual Machine



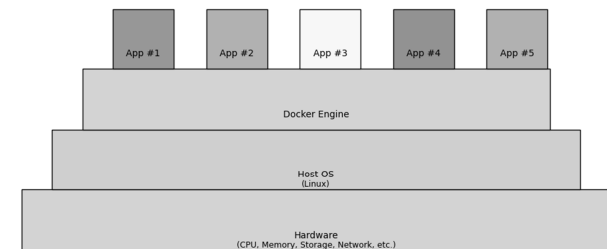
## Two Classic Problems in Deploying Software

- "It works on my machine" problem
- Dependency hell problem

## Container Technology

- Container technology is a method of packaging software applications and their dependencies into a standardized unit.
- The most popular container platform, known for its ease of use and large community. It provides tools for building, running, and managing containers.

## Container Technology (Cont.)



## Docker's Terminologies

- **Docker Engine:** The core component of Docker that manages containers, images, networks, and volumes.
- **Dockerfile:** A text file that contains a series of instructions on how to build a Docker image.
- **Docker Image:** A read-only template used to create Docker containers. Images contain the application code, runtime, libraries, and dependencies needed to run the application.

## Docker's Terminologies (cont.)

- **Docker Hub:** A cloud-based registry service for sharing Docker images. It allows users to search for, download, and upload images.
- **Docker Container:** A runnable instance of a Docker image. Containers are isolated from each other and the host system, ensuring that applications run consistently regardless of the environment.

## Docker's Terminologies (cont.)

- **Volume:** A mechanism for persisting data generated by and used by Docker containers.
- **Network:** Docker provides networking features to connect containers to each other and to the outside world.

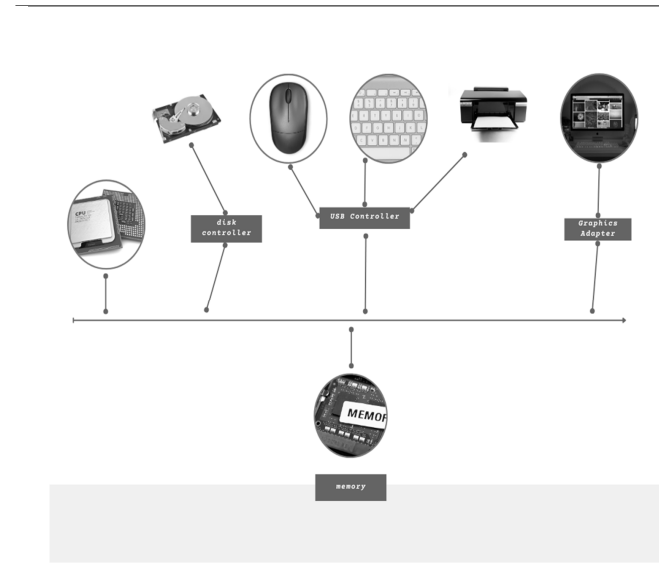
### Containerization & Virtualization

| Factor       | Containerization                            | Virtualization                           |
|--------------|---------------------------------------------|------------------------------------------|
| OS           | All containers run on the same host OS.     | Each OS for each VM.                     |
| Machine      | Do not need to simulate the entire machine. | An entire machine needs to be simulated. |
| Portability  | More portable                               | Less portable                            |
| Weight       | Light                                       | Heavy                                    |
| Booting Time | Fast                                        | Slow                                     |

# Computer Organization and OS

## Computer-System Operation

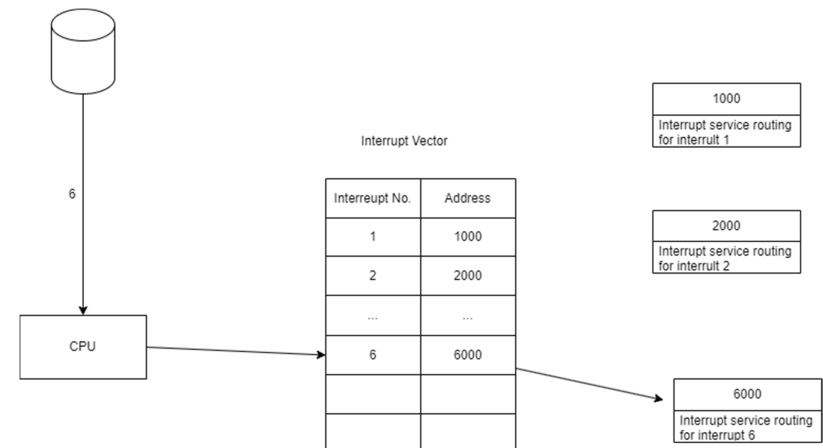
- I/O devices and the CPU can execute concurrently.
- Each device controller is in charge of a particular device type.
- Device controller informs CPU that it has finished its operation by causing an interrupt.



## Common Functions of Interrupts

- An operating system is interrupt driven.
- Interrupt architecture must save the address of the interrupted instruction.
- Interrupt passes control to the interrupt service routine generally by utilizing the interrupt vector, which stores the addresses of all the service routines..
- A trap is an interrupt generated by software, which can be triggered by either program error (exceptions) or request to the OS (through system calls).

## Common Functions of Interrupts (Cont.)



# Categories of OS

## Criteria

- How the data is entered into the system
- Number of users and Tasks, OS can support
- Special Purpose Systems
- Type of architecture of the computer

## How the data is entered into the system

- Batch
  - Single-Programmed Batch
  - Multi-Programmed Batch
- Interactive
- Hybrid

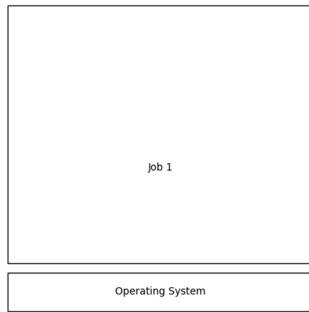
The jobs are executed sequentially, without any human intervention.

## Batch Systems

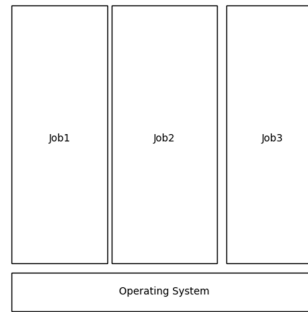
- The jobs are processed serially, without user interaction
- Initial primitive operating system.
  - Automatic job sequencing – automatic movement of control from one job to another.

## Single and Multi program Batch Systems

Single Program Batch



Multiprogram Batch



## Interactive Systems

- Introduced for programmers who needed to fix bugs in their programs quickly.
- Time-sharing software is needed.
- The CPU is split up between several jobs that are stored in memory and on disk. The CPU is only assigned to a job that is in memory.
- A job went back and forth between memory and the disk.
- On-line system is needed for users to access data and code.

## Hybrid Systems

- Combination of batch and interactive
- Executes and operates batch applications in the background during periods of low interactive workload

## Number of users & Tasks, OS can support

- Single-user single-tasking
  - MS DOS
- Single-user multi-tasking
  - Desktop (laptop) Operating Systems
- Multi-user multi-tasking
  - Server version of Operating Systems

## Special Purpose Systems

- Real Time Systems
  - Hard Real Time
  - Soft Real Time
- Embedded Systems

## Real Time Systems

- Hard real-time:
  - Secondary storage is either limited or nonexistent. Data is kept in short-term memory or read-only memory (ROM).
  - Incompatible with time-sharing applications and also not compatible with general-purpose operating systems.
  - Examples
    - VxWorks
    - RTX
- Soft real-time
  - Beneficial for applications that necessitate powerful operating-system capabilities, such as multimedia and virtual reality.

## Embedded Systems

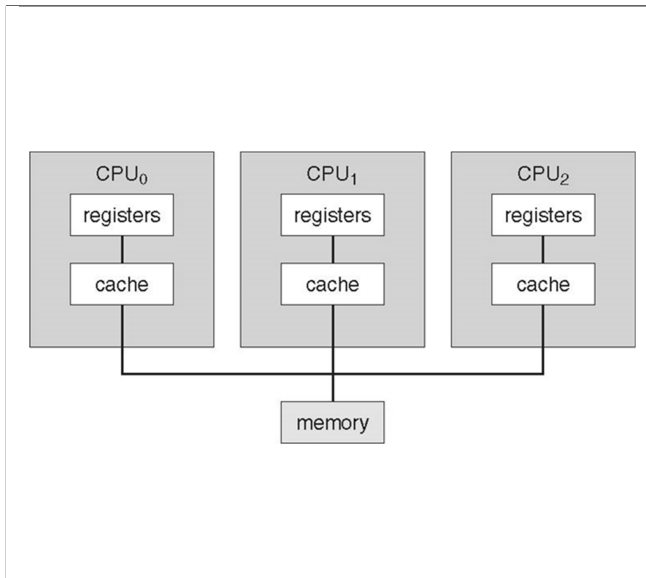
- An embedded system is a specialized computer system designed to perform a specific function or set of functions within a larger device or system.
- Unlike general-purpose computers (like your laptop or smartphone), embedded systems are dedicated to specific tasks and often operate with limited resources.

## Type of architecture of the computer

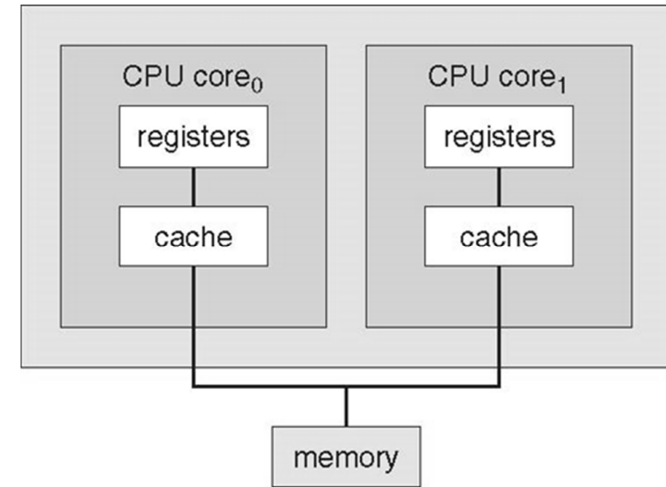
- Single Processor
- Parallel Systems (Multiprocessors, tightly-coupled)
- Distributed Systems

## Multiprocessors

Also known as parallel systems, tightly-coupled systems



## A Dual-Core CPU



## Distributed Systems

- A collection of independent computers that work together as a single, unified system.
- Each computer is a node, and they communicate and coordinate via a network.
- Loosely-coupled system – each processor has its own resources
- Requires networking infrastructure.
- OS for distributed systems may be divided into two types.
  - Network Operating Systems
    - Windows Server
    - Ubuntu Server
    - Windows 11?
    - MacOS??
  - Distributed Operating Systems
    - Sprite
    - Amoeba

## Comparison of Network and Distributed Operating Systems

| Network Operating System (NOS)                                         | Distributed Operating System (DO/S)                     |
|------------------------------------------------------------------------|---------------------------------------------------------|
| Resources owned by local nodes                                         | Resources owned by global system                        |
| Local resources managed by local operating system                      | Local resources managed by a global DO/S                |
| Access performed by a local operating system                           | Access performed by the DO/S                            |
| Requests passed from one local operating system to another via the NOS | Requests passed directly from node to node via the DO/S |

---

---

# Operating Systems

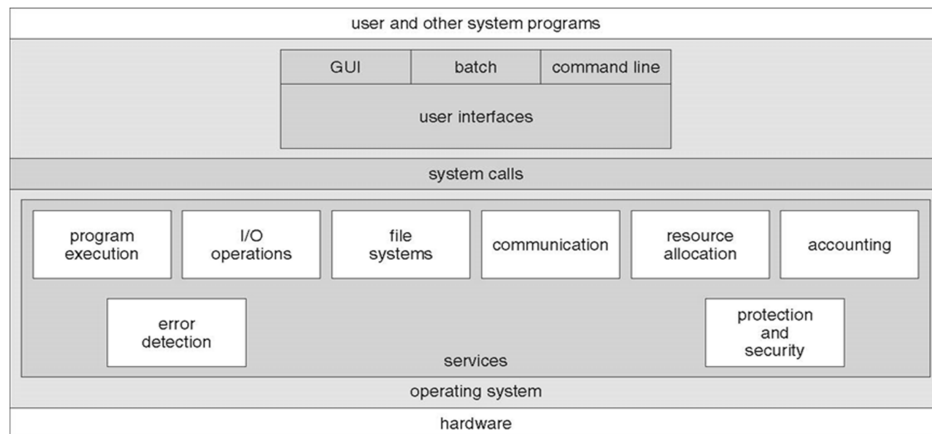
No. 2  
Sarun Intakosum

---

---

## OS Structure

## A View of Operating System Services



## Operating System Services

- User interface (UI)
    - Command-Line (CLI),
    - Graphics User Interface (GUI), Batch
  - Program execution -The system must be able to load a program into memory and execute it, terminating the execution either normally or abnormally to indicate an error.
  - I/O operations - A running program may require I/O (file or an I/O device)
  - File-system manipulation - Programs require the ability to do various file and directory operations, such as reading and writing files, creating and deleting them, searching for specific files, listing file information, and managing permissions.
-

## Operating System Services (Cont)

- Communications – Processes can communicate by sharing information, either on a single computer or amongst multiple computers connected through a network.
    - Communications may be via shared memory or through message passing
  - Error detection – The operating system must constantly watch over potential problems in both hardware and software.
    - The operating system should execute the necessary measures for each error category to guarantee accurate and uniform computing.
    - Debugging features can significantly improve the user's and programmer's capacity to utilize the system effectively.
- 

## Operating System Services (Cont)

- Resource allocation - When numerous processes operate concurrently, it is necessary to assign resources to each.
  - Accounting - Monitor the usage of computer resources by different users, including the amount and types of resources used.
  - Protection and security
    - Protection entails the implementation of measures to regulate and restrict all forms of access to system resources.
    - Ensuring the system's security against unauthorized individuals necessitates user authentication and also involves safeguarding external input/output devices from unauthorized access attempts
- 

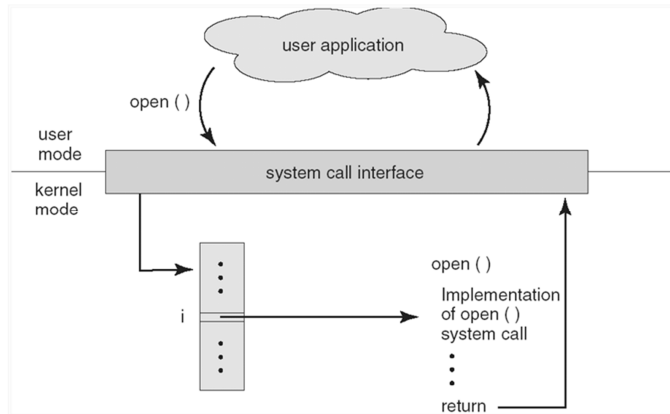
## User Operating System Interface - CLI

- Command Line Interface (CLI) or command interpreter allows direct command entry
    - Can be implemented in kernel
    - Can be implemented as external program - multiple flavors of shells
      - The main function of this program is to retrieve a command from the user and carry out its execution.
      - Sometimes commands built-in, sometimes just names of programs
- 

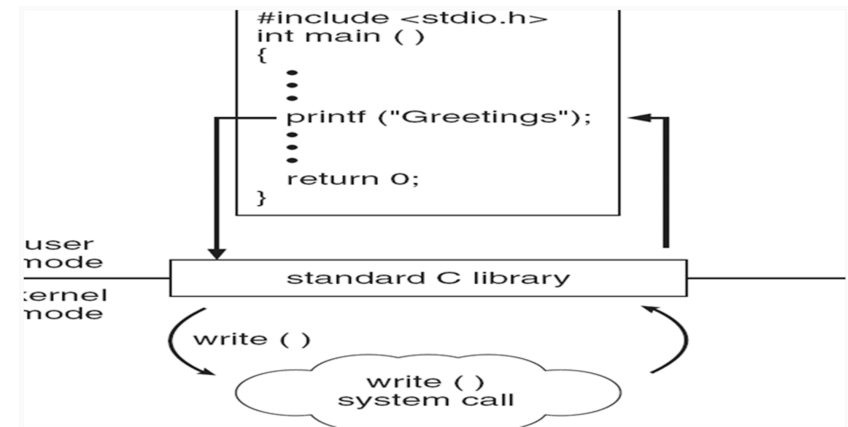
## System Calls

- Application programming interface (API) that allows access to the functions and services offered by the operating system.
  - Typically written in a high-level language (C or C++)
  - Typically, programs primarily employ **high-level** Application Program Interface instead of directly relying on OS system calls.
  - Three most common APIs are
    - Win32 API for Windows
    - POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS)
    - Java API for the Java virtual machine (JVM)
-

## API – System Call – OS Relationship



## Standard C Library Example



## Types of System Calls

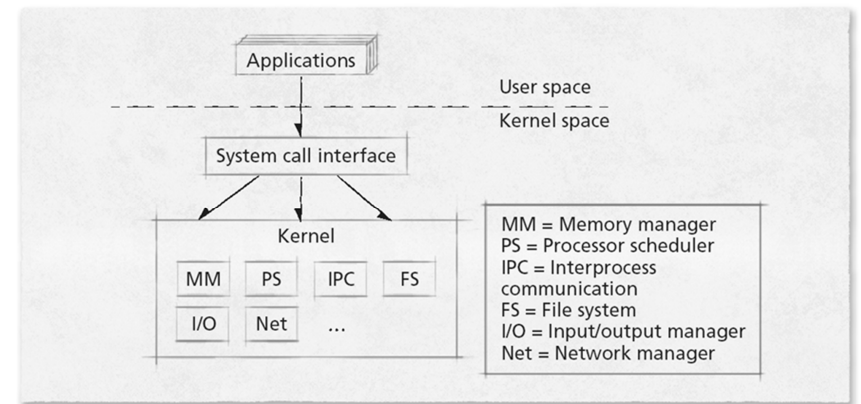
- Process control
- File management
- Device management
- Information maintenance
- Communications
- Protection

## Examples of Windows and Unix System Calls

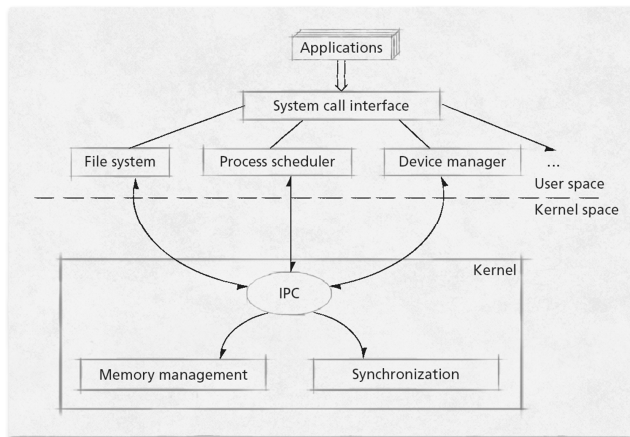
|                         | Windows                                                                             | Unix                                   |
|-------------------------|-------------------------------------------------------------------------------------|----------------------------------------|
| Process Control         | CreateProcess()<br>ExitProcess()<br>WaitForSingleObject()                           | fork()<br>exit()<br>wait()             |
| File Manipulation       | CreateFile()<br>ReadFile()<br>WriteFile()<br>CloseHandle()                          | open()<br>read()<br>write()<br>close() |
| Device Manipulation     | SetConsoleMode()<br>ReadConsole()<br>WriteConsole()                                 | ioctl()<br>read()<br>write()           |
| Information Maintenance | GetCurrentProcessID()<br>SetTimer()<br>Sleep()                                      | getpid()<br>alarm()<br>sleep()         |
| Communication           | CreatePipe()<br>CreateFileMapping()<br>MapViewOfFile()                              | pipe()<br>shmget()<br>mmap()           |
| Protection              | SetFileSecurity()<br>InitializeSecurityDescriptor()<br>SetSecurityDescriptorGroup() | chmod()<br>umask()<br>chown()          |

## Kernel Architectures

## Monolithic Architecture



## Microkernel Architecture



# Operating Systems

No. 3

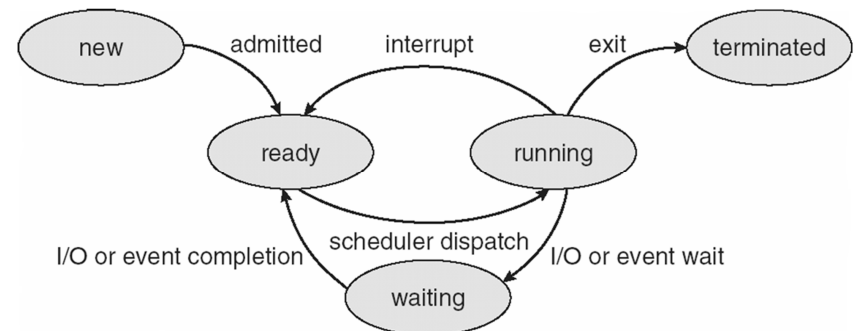
Sarun Intakosum

## Process Concept

## Process Concept

- Program: An inactive unit, such as a file stored on a disk.
- Process – a program in execution;
  - An active entity, which requires a set of resources. It has its own data such as:
    - program counter
    - stack
    - data section
  - A single instance of an executable program
  - Used in Time-Sharing System, also called Task
  - Batch system – jobs

## Diagram of Process State



## Seven-State Process Model

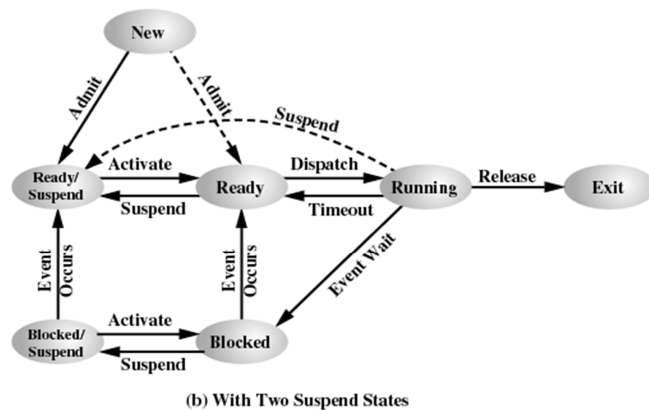
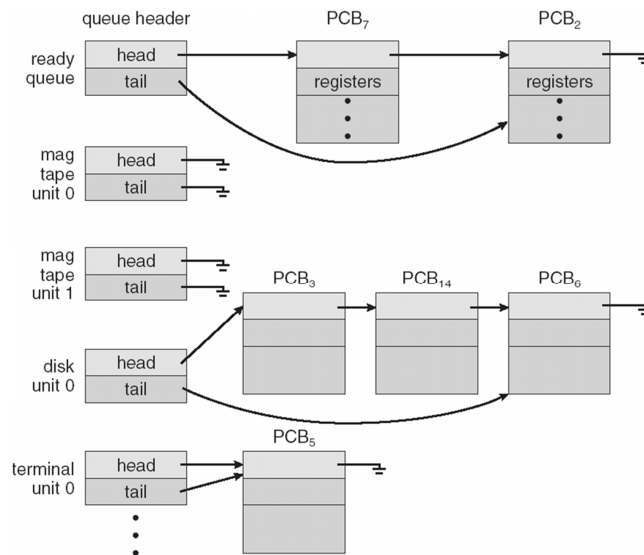


Figure 3.9 Process State Transition Diagram with Suspend States

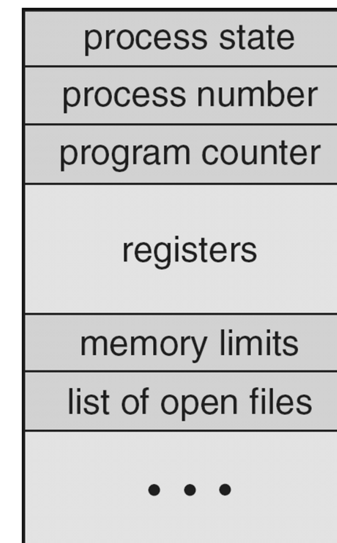
## Process Scheduling Queues

- **Job queue** – set of all processes in the system
- **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
- **Device queues** – set of processes waiting for an I/O device
- Processes migrate among the various queues

## Ready Queue And Various I/O Device Queues



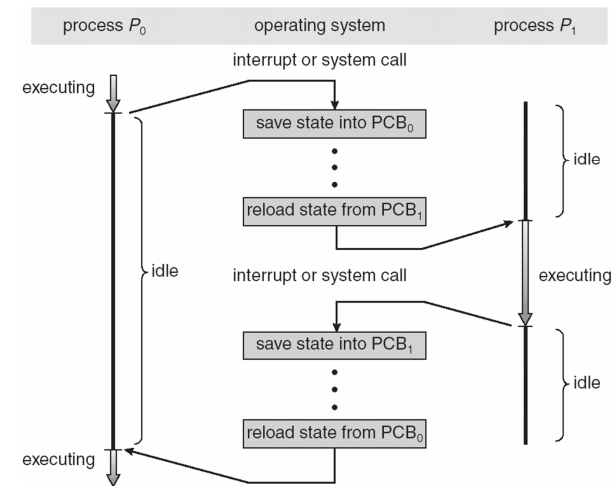
## Process Control Block (PCB)



## Context Switch

- When CPU switches to another process, the system must save the state of the old process and load the saved state for the new process via a context switch
- Context of a process represented in the PCB
- Context-switch time is overhead; the system does no useful work while switching
- Time dependent on hardware support

## Context Switch



## Dispatcher

- Dispatcher module gives control of the CPU to the process selected by the short-term scheduler; this involves:
  - switching context
  - switching to user mode
  - jumping to the proper location in the user program to restart that program
- **Dispatch latency** – time it takes for the dispatcher to stop one process and start another running

## Schedulers

- **Long-term scheduler** (or job scheduler) – selects which processes should be brought into the ready queue
- **Short-term scheduler** (or CPU scheduler) – selects which process should be executed next and allocates CPU

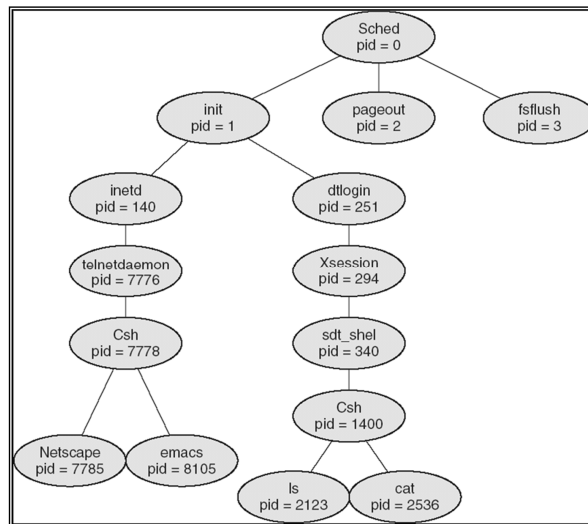
## Schedulers (Cont)

- Short-term scheduler is invoked very frequently (milliseconds)  $\Rightarrow$  (must be fast)
- Long-term scheduler is invoked very infrequently (seconds, minutes)  $\Rightarrow$  (may be slow)
- The long-term scheduler controls the *degree of multiprogramming*
- Processes can be described as either:
  - **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts
  - **CPU-bound process** – spends more time doing computations; few very long CPU bursts

## Process Creation

- The parent process spawns child processes, which in turn generate further processes, resulting in a hierarchical structure of processes known as a process tree.
- UNIX examples
  - **fork** system call creates new process

## A tree of processes on a typical Solaris



## fork()

- ```
#include <sys/types.h>
#include <unistd.h>
pid_t fork(void);
```
- Creates a child process by making a copy of the parent process.
- Both the child *and* the parent continue running.

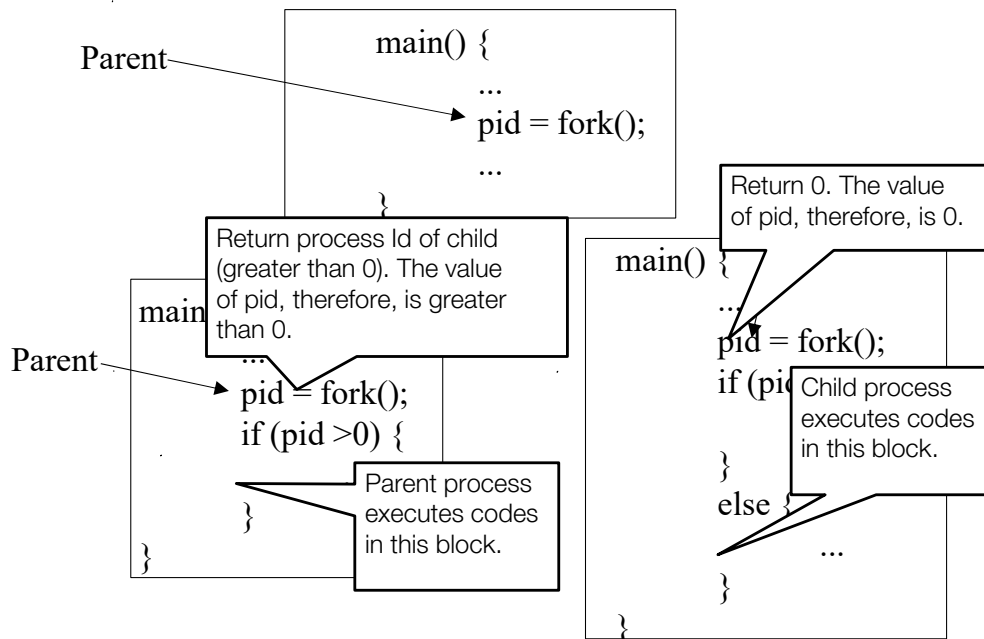
## fork()

- `#include <sys/types.h>`  
`#include <unistd.h>`  
`pid_t fork(void);`
- `fork()` is a fundamental operation in Unix-like systems (including Linux and macOS).
- It's a system call that allows a running process to create a copy of itself.
- The original process is called the **parent process**, and the new copy is called the **child process**.
- **Both the child *and* the parent continue running.**
- **fork\_example.c**

## Example of fork()

```
/* fork_example.c */
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
    pid_t pid;
    int i = 0;
    pid = fork();
    if (pid > 0) { /* parent */
        i = 1;
        printf("I am parent i = %d\n", i);
    }
    else { /* child */
        i = 2;
        printf("I am child i = %d\n", i);
    }
    return 0;
}
```

## Diagram for fork() example



## Process IDs (pids)

- `pid = fork();`
- In the child: `pid == 0`;  
In the parent: `pid ==` the process ID of the child.
- If the value of `pid` is negative, it indicates that the creation of a child process was unsuccessful.
- The application can utilize this difference in the value of `pid` to execute distinct actions in the parent and child processes.

## Things to Note

- i is copied between parent and child.
- Output interleaving is inherently non-deterministic, meaning that the output cannot be determined solely by examining the code.

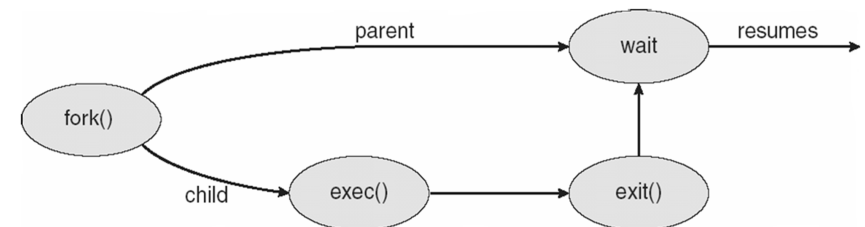
## exec()

- exec() is family of functions in Unix-based operating systems
- exec() replaces the current process image with a new process image.
- Essentially, it transforms the existing process into a different program.

## Fork and exec()

```
int main()
{
    pid_t pid;
    /* fork another process */
    pid = fork();
    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        exit(-1);
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait (NULL);
        printf ("Child Complete");
        exit(0);
    }
}
```

## Process Creation



## Process Termination

- The process terminates by executing its final statement and requesting the operating system to destroy it (exit).
  - Output data from child to parent (via **wait**)
  - Process' resources are deallocated by operating system
- Parents have the ability to terminate the execution of child processes, often known as aborting them.
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - If parent is exiting
    - 4 Some operating system do not allow child to continue if its parent terminates
      - All children terminated - *cascading termination*

## Process in Java

- On most systems, the JVM is considered as an ordinary application running as a separate process on the host operating system.
- Java does not support process model, which would allow the JVM to create several processes within the same virtual machine.
- It is possible to create a process external to the JVM, however, by using the ProcessBuilder class, which allows a Java program to specify a process that is native to the OS.
- JavaProcess.java

# Operating Systems

No. 4

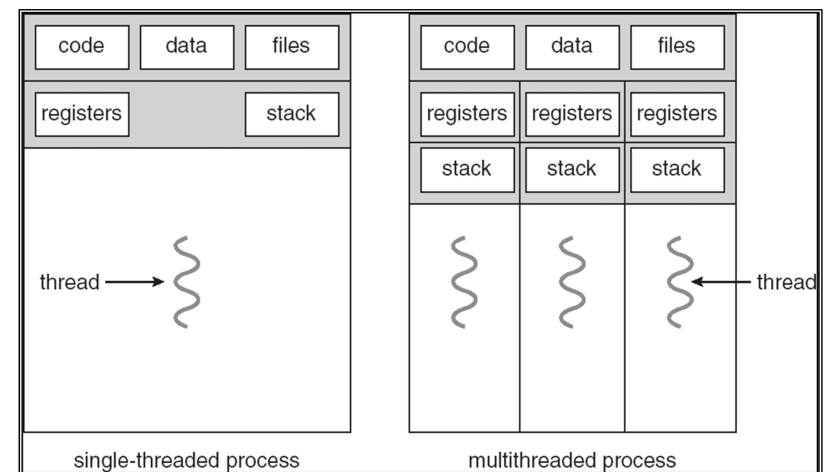
Sarun Intakosum

## Thread

### Threads Concepts

- **Thread:** A portion of a program that can run independently of other portions.
  - A thread is a sequence of instructions that runs independently within a process, with its own program counter, system registers, and stack.
  - Thread is also called **lightweight process**.
  - Thread becomes the element that uses the CPU and is scheduled for execution
  - Swapping threads is less time consuming than swapping processes
- Multithreaded applications programs can have several threads running at one time with the same or different priorities

### Single and Multithreaded Processes



# Categories of OS

- Single-process single-threaded
  - DOS
- Single-Process multi-threaded
  - JVM *↓ process can have many thread.*
- Multiprocessing single-threaded
  - Tradition UNIX
- Multiprocessing multi-threaded
  - Modern Operating Systems => Windows, Linux, MacOS

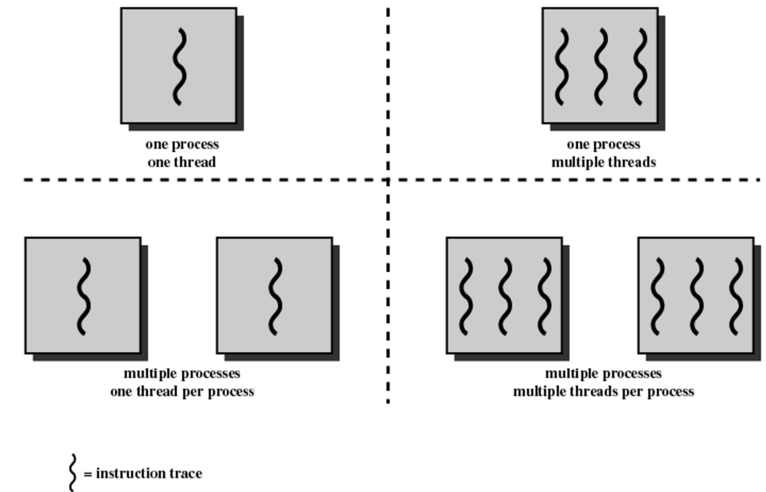
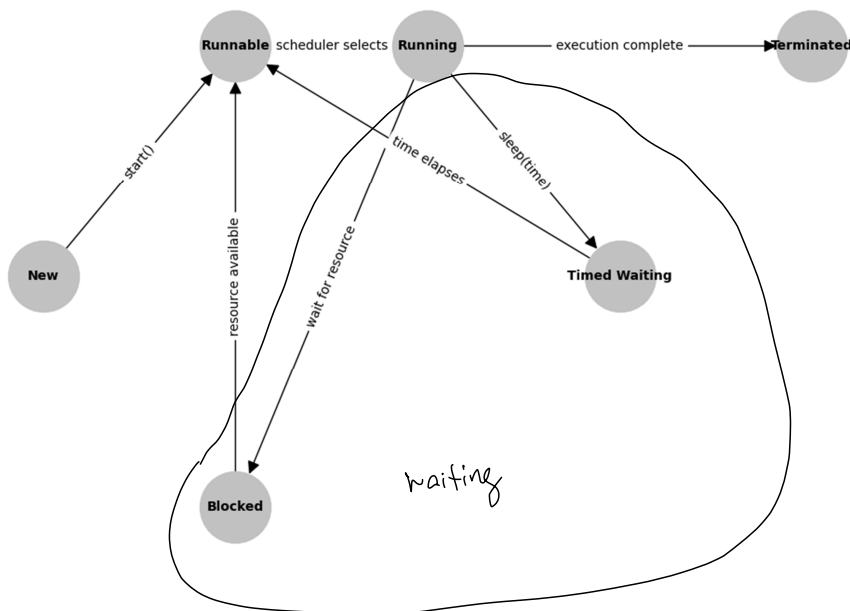


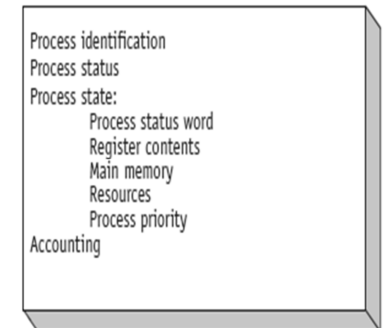
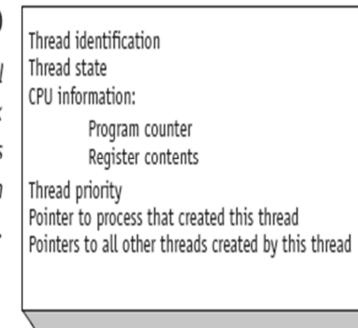
Figure 4.1 Threads and Processes [ANDE97]

# Thread States



# Thread Control Block

(figure 6.7)  
Comparison of a typical  
Thread Control Block  
(TCB) vs. a Process  
Control Block (PCB) from  
Chapter 4.



## Types of Threads

There are two general types of threads:

- User-level threads (ULT) *handle at User level*
- Kernel-level threads (KLT).

## User Threads

- Thread management done by user-level threads library
- Three primary thread libraries:
  - POSIX Pthreads
  - Win32 threads
  - Java threads
    - Generally implemented using a thread library available on host OS.
- This library enables the program to call the necessary functions for managing threads, including establishing, suspending, and terminating them.

*JVM [ windows - win32, linux - posix ]*

## Kernel Threads

- Supported by the Kernel
- Examples
  - Modern operating systems

*mapping between user level → kernel level*

## Multithreading Models

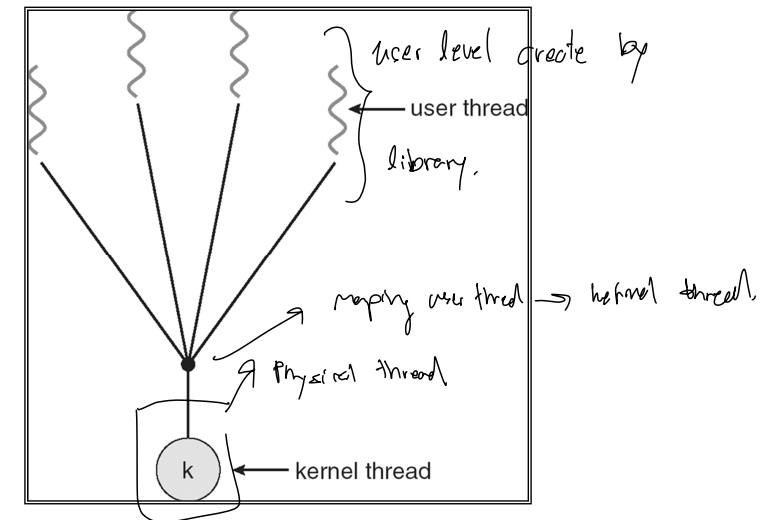
- Many-to-One *Many user → One thread*
- One-to-One
- Many-to-Many

## Many-to-One

- Many user-level threads mapped to single kernel thread
- Examples:
  - Early Unix System

user  $\rightarrow$  library  
 kernel  $\rightarrow$  OS

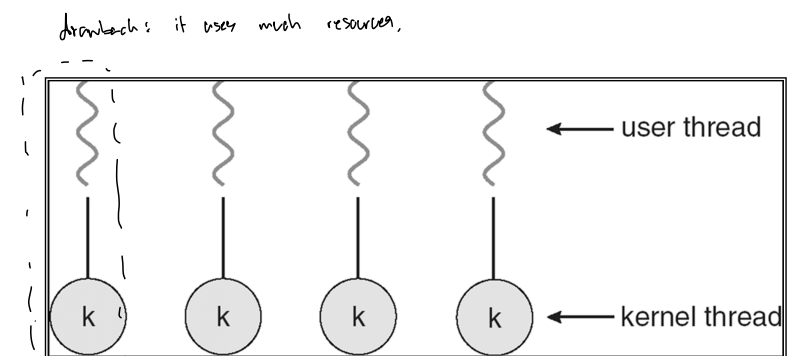
## Many-to-One Model



## One-to-One

- Each user-level thread maps to kernel thread
- Examples
  - Windows starting from NT
  - Linux
  - Solaris 9 and later

## One-to-one Model

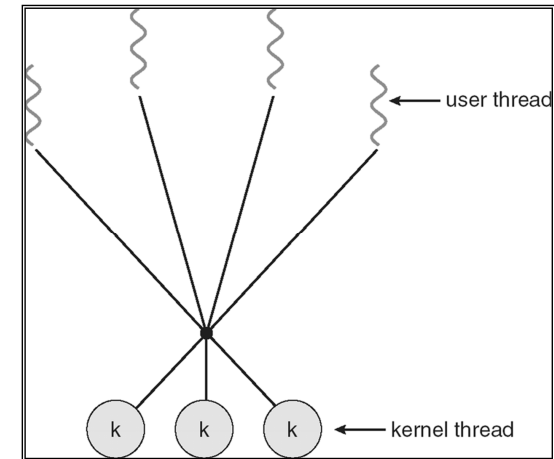


Suppose if the thread do nothing, it will just waiting the resource.

benefit: easy to manage.

## Many-to-Many Model

- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads
- Examples
  - Solaris prior to version 9
  - Tru64 UNIX



## Pthreads

```
int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* the thread */
main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of attributes for the thread */
    pthread_attr_init(&attr); /* get the default attributes */
    /* create the thread */
    pthread_create(&tid, &attr, runner, argv[1]);
    /* now wait for the thread to exit */
    pthread_join(tid, NULL); ← wait for the process id.
    printf("sum = %d\n", sum);
}

void *runner(void *param) {
    int upper = atoi(param);
    int i; convert string to int.
    sum = 0;
    if (upper > 0) {
        for (i = 1; i <= upper; i++)
            sum += i;
    }
    pthread_exit(0);
}
```

## Java Threads

- Java threads are managed by the JVM
- Java threads may be created by:
  - Extending Thread class
  - Implementing the Runnable interface

## Extending the Thread Class

```
class Worker1 extends Thread
{
    public void run() {
        System.out.println("I Am a Worker Thread");
    }
}

public class First
{
    public static void main(String args[]) {
        Worker1 runner = new Worker1();
        runner.start();

        System.out.println("I Am The Main Thread");
    }
}
```

## The Runnable Interface

```
public interface Runnable
{
    public abstract void run();
}
```

## Implementing the Runnable Interface

```
class Worker2 implements Runnable
{
    public void run() {
        System.out.println("I Am a Worker Thread ");
    }
}

public class Second
{
    public static void main(String args[]) {
        Runnable runner = new Worker2();
        Thread thrd = new Thread(runner);
        thrd.start();

        System.out.println("I Am The Main Thread");
    }
}
```

## Joining Threads

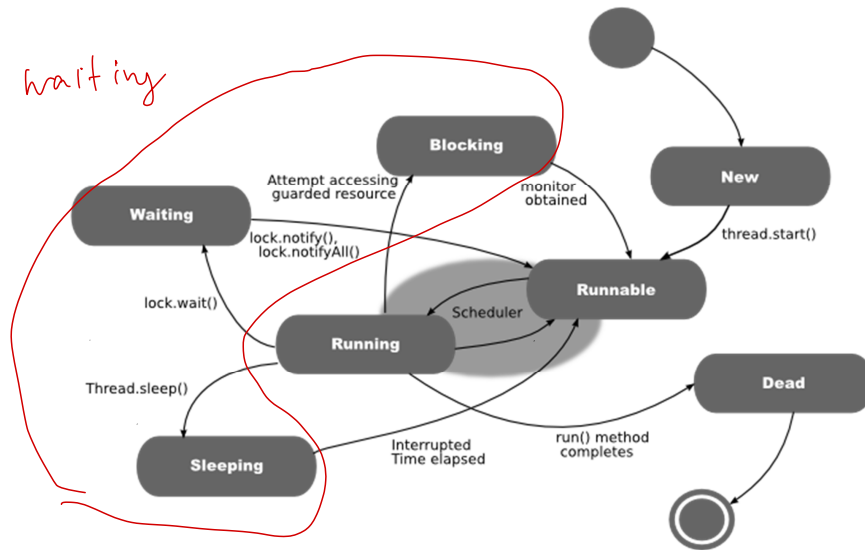
```
class JoinableWorker implements Runnable
{
    public void run() {
        System.out.println("Worker working");
    }
}

public class JoinExample
{
    public static void main(String[] args) {
        Thread task = new Thread(new JoinableWorker());
        task.start();

        try { task.join(); }
        catch (InterruptedException ie) { }

        System.out.println("Worker done");
    }
}
```

## Java Thread States



<https://www.pngegg.com/en/png-tvayp>

## Thread Cancellation

- Terminating a thread before it has finished
- Two general approaches:
  - Asynchronous cancellation** terminates the target thread immediately
  - Deferred cancellation** allows the target thread to periodically check if it should be cancelled

*pthread* {  
 ↓  
 default is deferred, consider suitable.

*pthread\_t* - set cancel state (PTHREAD\_CANCEL\_DISABLE)

## Thread Cancellation

```
Thread thrd = new Thread (new InterruptibleThread());
Thrd.start();
```

...

```
// now interrupt it
Thrd.interrupt();
```

## Thread Cancellation (Cont.)

```
public class InterruptibleThread implements Runnable
{
    public void run() {
        while (true) {
            /**
             * do some work for awhile
             */

            if (Thread.currentThread().isInterrupted()) {
                System.out.println("I'm interrupted!");
                break;
            }
        }

        // clean up and terminate
    }
}
```

bring one that available

## Thread Pools

- **Instantiate several threads** in a pool and have them remain idle, waiting for tasks to be assigned.
- Advantages:
  - Typically, handling a request using an existing thread is more efficient than creating a new one.
  - Enables the application(s) to allocate the number of threads based on the size of the pool.

if finish it will go back to the pool.

## Thread Pools

- Java provides 3 thread pool architectures:
  1. **Single thread executor** - pool of size 1.
    - `static ExecutorService newSingleThreadExecutor()`
  2. **Fixed thread executor** - pool of fixed size.
    - `static ExecutorService newFixedThreadPool(int nThreads)`
  3. **Cached thread pool** - pool of unbounded size
    - `static ExecutorService newCachedThreadPool()`

Task.java  
NonTPEExample.java  
TPEExample.java

## CPU Scheduling

## CPU Scheduler

- Chooses from the processes in memory that are ready to run, and assigns the CPU to one of them
- **nonpreemptive** OS can interrupt the process.
- **preemptive**
  - ↓
  - OS can interrupt the running process.

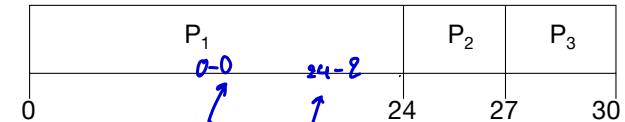
## Scheduling Algorithm Optimization Criteria

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

## First-Come, First-Served (FCFS) Scheduling

non-preemptive Algo  $\rightarrow$  First come, first served.

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
| $P_1$   | 0.0          | 24         |
| $P_2$   | 2.0          | 3          |
| $P_3$   | 4.0          | 3          |



- Waiting time for  $P_1 = 0$ ;  $P_2 = 22$ ;  $P_3 = 23$
- Average waiting time:  $(0 + 22 + 23)/3 = 15$

$$\left( \frac{0 + 22 + 23}{3} \right) \approx 15$$

turn around time = finish time - arrival time

$$TP_1 = 24 - 0 = 24$$

$$TP_2 = 27 - 2 = 25$$

$$TP_3 = 30 - 4 = 26$$

waiting time = turnaround time - burst time

$$WP_1 = 24 - 24 = 0$$

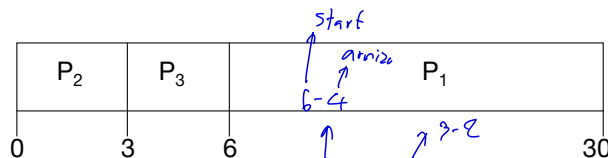
$$WP_2 = 25 - 3 = 22$$

$$WP_3 = 26 - 3 = 23$$

## FCFS Scheduling (Cont)

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
|---------|--------------|------------|

- 1  $P_2$  0.0 3
- 2  $P_3$  2.0 3
- 3  $P_1$  4.0 24



- Waiting time for  $P_1 = 2$ ;  $P_2 = 0$ ;  $P_3 = 1$
- Average waiting time:  $(2 + 0 + 1)/3 = 1$
- Much better than previous case
- Convoy effect short process behind long process

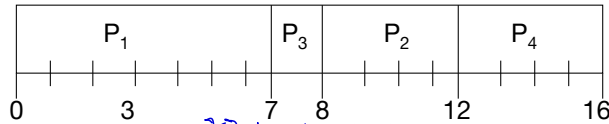
## Shortest-Job-First (SJF) Scheduling

- Assign a value representing the duration of the next CPU burst to each process.
- Utilize these lengths to efficiently schedule the process with the shortest time.
- SJF is optimal – gives minimum average waiting time for a given set of processes
  - It can be challenging to determine the length of the upcoming CPU request

## Example of SJF

| Process        | Arrival Time | Burst Time |
|----------------|--------------|------------|
| P <sub>1</sub> | 0.0          | 7          |
| P <sub>2</sub> | 2.0          | 4          |
| P <sub>3</sub> | 4.0          | 1          |
| P <sub>4</sub> | 5.0          | 4          |

- SJF scheduling chart



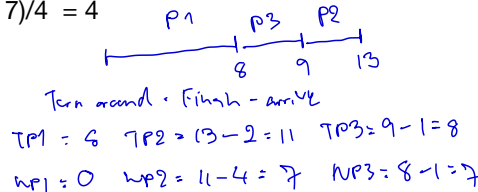
- Average waiting time =  $(0 + 6 + 3 + 7)/4 = 4$

$$TP_1 = 7 - 0 = 7$$

$$TP_2 = 12 - 2 = 10$$

$$TP_3 = 8 - 4 = 4$$

$$TP_4 = 16 - 5 = 11$$



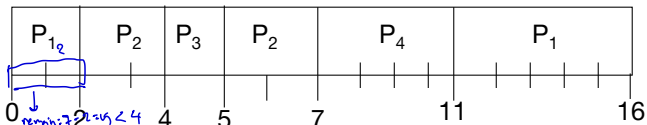
## Shortest Remaining Time First (SRTF)

- Preemptive version of the SJF algorithm
- Assigning the processor to the job that is closest to completion
  - If a newer job in the READY queue has a shorter time to completion, it can preempt the current job
- Cannot be implemented in interactive system
  - Requires advance knowledge of the CPU time required to finish each job
- SRTF has more overhead than SJF
  - The operating system tracks the CPU time for all jobs in the READY queue and handles context switching.

## Example of SRTF

| Process        | Arrival Time | Burst Time |
|----------------|--------------|------------|
| P <sub>1</sub> | 0.0          | 7          |
| P <sub>2</sub> | 2.0          | 4          |
| P <sub>3</sub> | 4.0          | 1          |
| P <sub>4</sub> | 5.0          | 4          |

- SJF (preemptive)



- Average waiting time =  $(9 + 1 + 0 + 2)/4 = 3$

Turn around = Finish - Arrive

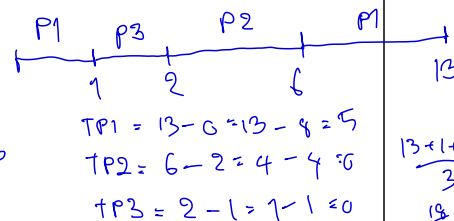
$$TP_1 = 16 - 0 = 16$$

$$TP_2 = 7 - 2 = 5$$

$$TP_3 = 5 - 4 = 1$$

$$TP_4 = 11 - 5 = 6$$

$$\left. \begin{array}{l} \text{Waiting time} = \text{Turn around} - \text{burst time} \\ WP_1 = 16 - 7 = 9 \\ WP_2 = 5 - 4 = 1 \\ WP_3 = 1 - 1 = 0 \\ WP_4 = 6 - 4 = 2 \end{array} \right\} \frac{(9 + 1 + 0 + 2)}{4} = 3$$



## Priority Scheduling

- Each process is assigned a priority value, typically an integer. The priority can be determined based on various criteria, such as the importance of the task, user preference, or resource requirements.
- The CPU is allocated to the process with the highest priority
- Algorithms
  - Preemptive
  - Non-preemptive
- SJF is a priority scheduling where priority is the predicted next CPU burst time
- Problem  $\equiv$  **Starvation** - low priority processes may never execute
- Solution  $\equiv$  **Aging** - as time progresses increase the priority of the process

longer time  $\rightarrow$  will increase the priority of the process.

$$\frac{13 + 1 + 4}{3} = 6 \#$$

10.)

|    |   |   |
|----|---|---|
| P1 | 8 | 7 |
| P2 | 4 |   |
| P3 | 1 |   |

$\overbrace{\quad}^{P1}$   
1

## fine sharing Round Robin (RR)

- Every process is allocated a small amount of CPU time, typically ranging from 10 to 100 milliseconds, known as a time quantum.
- Once this time has passed, the process is interrupted and placed at the end of the ready queue
- If the ready queue contains  $n$  processes and the time quantum is  $q$ , each process is allocated no more than  $q$  time units at a time. No process waits more than  $(n-1)q$  time units.
- Performance
  - $q$  large  $\Rightarrow$  FIFO
  - $q$  small  $\Rightarrow q$  must be large with respect to context switch, otherwise overhead is too high

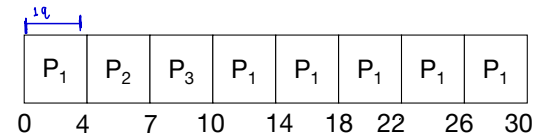
Ex  $P_1, P_2, P_3, P_4$

$n=4$   
 $q = 10 \text{ ms}$      $(n-1)q$   
 $(4-1)(10 \text{ ms}) = 30 \text{ ms}$

## Example of RR with Time Quantum = 4

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
| $P_1$   | 0.0          | 24         |
| $P_2$   | 2.0          | 3          |
| $P_3$   | 4.0          | 3          |

- The Gantt chart is:



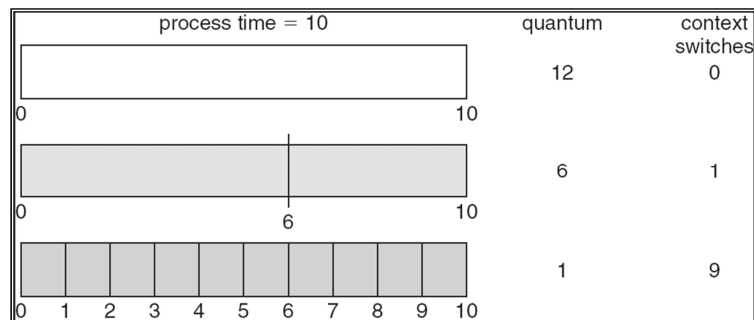
- Typically, higher average turnaround than SJF, but better response

$T_{\text{turn}} = \text{Finish} - \text{arrive}$      $T_{\text{turn}} = \text{wait} + \text{burst}$

$TP_1 = 30 - 0 = 30$   
 $TP_2 = 7 - 2 = 5$   
 $TP_3 = 10 - 4 = 6$

$\text{wait} = T_{\text{turn}} - \text{burst}$   
 $WP_1 = 30 - 24 = 6$   
 $WP_2 = 5 - 3 = 2$   
 $WP_3 = 6 - 3 = 3$

## Time Quantum and Context Switch Time



## Multilevel Queue Scheduling

- Multilevel Queue Scheduling is a scheduling algorithm that partitions the ready queue into several separate queues.
- Each queue can have its own scheduling algorithm. For Examples
  - foreground – RR  $\rightarrow$  interact with user
  - background – FCFS  $\rightarrow$  batch
- This method is used when processes can be easily categorized into different classes with different scheduling needs.
- No Movement Between Queues: Processes do not move between queues. Once assigned to a queue, a process stays there.

## Multilevel Queue Scheduling (Cont.)

- Scheduling must be done between the queues
  - Fixed priority scheduling; (i.e., serve all from foreground then from background). Possibility of starvation.
  - Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR 20% to background in FCFS

avoid starvation

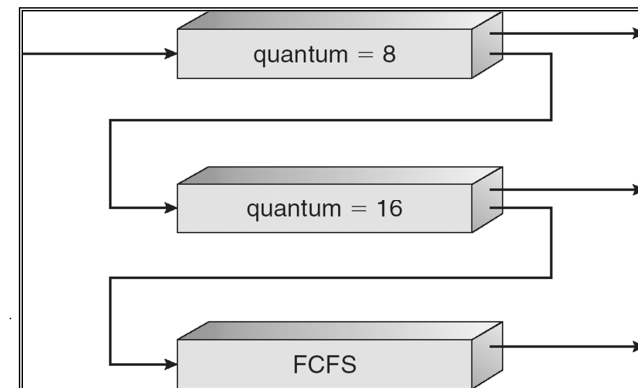
## Multilevel Feedback Queue Scheduling

- A process can move between the various queues; aging can be implemented this way
- Multilevel-feedback-queue scheduler defined by the following parameters:
  - number of queues
  - scheduling algorithms for each queue
  - method used to determine when to upgrade a process
  - method used to determine when to demote a process
  - method used to determine which queue a process will enter when that process needs service

## Example of Multilevel Feedback Queue

- Three queues:
  - $Q_0$  – RR with time quantum 8 milliseconds
  - $Q_1$  – RR time quantum 16 milliseconds
  - $Q_2$  – FCFS
- Scheduling
  - A new job enters queue  $Q_0$  which is served FCFS. When it gains CPU, job receives 8 milliseconds. If it does not finish in 8 milliseconds, job is moved to queue  $Q_1$ .
  - At  $Q_1$  job is again served FCFS and receives 16 additional milliseconds. If it still does not complete, it is preempted and moved to queue  $Q_2$ .

## Multilevel Feedback Queues



## Java Thread Scheduling

- Java specification does not specify the standard scheduling.
- It depends on the implementation of each JVM.
- Do not rely on the scheduling algorithm for the correctness of your program.

## Java Threads Scheduling (Cont.)

- Java typically employs a preemptive and priority-based scheduling method.
- Every Java thread possesses a priority. The thread with the greatest priority is selected as the first choice.
- When multiple threads with the same priority await the CPU, the scheduler selects one to execute randomly.

## Java Threads Scheduling (Cont.)

- The chosen thread runs until one of the following conditions is true:
  - A higher priority thread becomes runnable.
  - It yields, or its run method exits.
  - On systems that support time-slicing, its time slot has expired.

## Java Threads Scheduling (Cont.)

- Priority is in the range of 1-10.
- The default priority is 5.
- Method `setPriority()` is used to set a thread priority.
  - `myThread.setPriority(6);`
- A thread can yield control of the CPU using the `yield()` method.
  - Java can then choose another thread with the same priority to run.

